Ход выполнения лабораторной работы №7

Пример 1 — Парсинг через int.Parse

```
string a = Console.ReadLine()!;
int b = int.Parse(a);
Console.WriteLine(b);
```

При вводе числа 42 программа успешно его распарсила и вывела. При вводе строки hello программа упала с ошибкой:

Unhandled exception. System.FormatException: The input string 'hello' was not in a correct format.

int.Parse не обрабатывает некорректный ввод — если строка не является числом, программа аварийно завершается.

```
valentinaadvahova@Valentinas-MacBook-Air MySecondProject % dotnet run
=== 1 - int.Parse ===
42
42
valentinaadvahova@Valentinas-MacBook-Air MySecondProject % dotnet run

=== 1 - int.Parse ===
hello
Unhandled exception. System.FormatException: The input string 'hello' was not in a correct format.
   at System.Number.ThrowFormatException[TChar](ReadOnlySpan`1 value)
   at System.Int32.Parse(String s)
   at Program.<Main>$(String[] args) in /Users/valentinaadvahova/Projects/univer/MySecondProject/Program.cs:line 3
```

Пример 2 — Парсинг через int.TryParse

```
string a = Console.ReadLine()!;
int b;
bool success = int.TryParse(a, out b);
Console.WriteLine(success);
Console.WriteLine(b);
```

При вводе 42 вывелось True и 42. При вводе hello вывелось False и 0. int.TryParse не падает при некорректном вводе — возвращает false в success и 0 в b. Это безопасный способ парсинга.

```
valentinaadvahova@Valentinas-MacBook-Air MySecondProject % dotnet run
=== 2 - int.TryParse ===
42
True
42
valentinaadvahova@Valentinas-MacBook-Air MySecondProject % dotnet run
=== 2 - int.TryParse ===
hello
False
0
```

Пример 3 — Передача int переменной

int a = 0;
F(a);
Console.WriteLine(a);
static void F(int a) { a = 1; }

При вызове F создаётся локальная копия переменной a. Изменение копии внутри функции не влияет на оригинал. Вывелось: **0**

Пример 4 — Передача out параметра

int a = 0;
F(out a);
Console.WriteLine(a);
static void F(out int b) { b = 1; }

При out передаётся адрес переменной a, а не её копия. Функция записывает 1 напрямую в ячейку памяти a. Вывелось: **1**

Пример 5 — Совпадение имён с out параметром

int a = 0;
F(out a);
Console.WriteLine(a);
static void F(out int a) { a = 1; }

Имена переменных внутри и снаружи функции не имеют значения — передаётся адрес. Результат идентичен примеру 4. Вывелось: **1**

Примеры 6, 7, 8, 9 — Ошибки компиляции

Эти примеры не компилируются по следующим причинам: несовместимость типов при передаче out параметра (6, 7, 8), или неполная инициализация всех полей структуры (9).

Пример 10 — Инициализация всех полей struct через out

static void F(out A b) { b.f1 = 1; b.f2 = 2; }

Все поля структуры инициализированы — компилируется успешно. Вывелось: **1, 2**

Пример 11 — Инициализация всего struct через out

static void F(out A b) { b = new() { f1 = 1, f2 = 2 }; }

Структура инициализируется целиком через new. Вывелось: **1, 2**

Пример 12 — Возвращаемое значение при наличии out

static int F(out int b) { b = 1; return 2; }

Возвращаемое значение 2 никуда не сохраняется. В a записывается 1 через out. Вывелось: **1**

Пример 13 — Сохранение возвращаемого значения

b = F(out a);

a получает значение через out — **1**. b получает возвращаемое значение — **2**. Вывелось: **1, 2**

Пример 14 — Несколько out параметров

static void F(out int c, out int d) { c = 1; d = 2; }

Можно передавать несколько out параметров одновременно. Вывелось: **1, 2**

Пример 15 — Передача поля struct через out

F(out p.X);

Можно передавать поле структуры как out параметр — передаётся адрес поля. В p.X записывается 2. Вывелось: **2**

```
valentinaadvahova@Valentinas-MacBook-Air MySecondProject % dotnet run
=== 3 ===
0
=== 4 ===
1
=== 5 ===
1
=== 10 ===
1
2
=== 11 ===
1
2
=== 12 ===
1
=== 13 ===
1
2
=== 14 ===
1
2
=== 15 ===
2
```

Практика — Ввод 3 чисел больше 2

Была написана программа которая запрашивает 3 числа больше 2 и выводит их произведение. Для безопасного парсинга использовался int.TryParse. При некорректном вводе или числе меньше/равном 2 программа запрашивает ввод повторно через бесконечный цикл while (true) с continue. При вводе чисел 3, 4, 5 вывелось: **Произведение: 60**

```
valentinaadvahova@Valentinas-MacBook-Air MySecondProject % dotnet run
Введите 3 числа больше 2:
Введите число: hello
Ошибка: введите число!
Введите число: 1
Ошибка: число должно быть больше 2!
Введите число: 2
Ошибка: число должно быть больше 2!
Введите число: 3
Введите число: 4
Введите число: 5
Произведение: 60
```

Вывод

В ходе лабораторной работы были изучены способы парсинга строк в числа — int.Parse и безопасный int.TryParse. Был изучен механизм out параметров, который позволяет функции записывать значения напрямую в переменные вызывающего кода через передачу адреса. Была написана практическая программа с валидацией ввода.